

New Models, Services, and Frameworks

Gemma 3 by Google

<https://blog.google/technology/developers/gemma-3/>

It's four models: 1B, 4B, 12B, 27B, with the larger three ones multimodal. Context length 128k, function calling & structured outputs

The authors claim multilinguality – support of as many as 140 languages. (At least for some of them, Gemma 3 works significantly better than GPT-4o.)

At ChatBot Arena, Gemma 3 27B is second only to DeepSeek-R1 among open-source models. And a great news for LLM Engineers: a 27B model is optimized to fit on one H100/A100 GPU @ bf16!

Native multimodality with Gemini Flash 2

For some time already, we've been discussing LLMs that generate images. Usually, the idea is to apply image de-tokenizer to some "visual tokens" generated by the autoregressive transformer

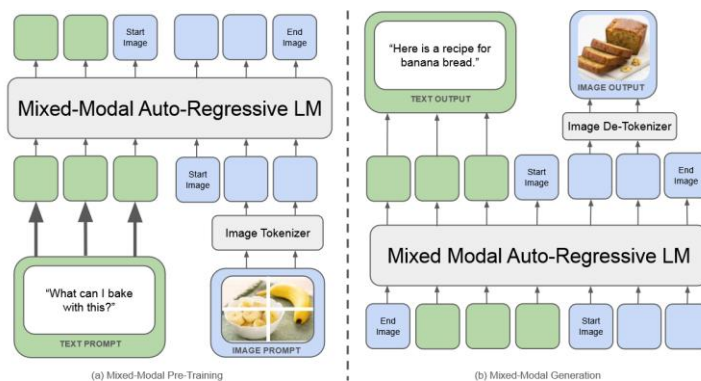


Figure 1 Chameleon represents all modalities — images, text, and code, as discrete tokens and uses a uniform transformer-based architecture that is trained from scratch in an end-to-end fashion on ~10T tokens of interleaved mixed-modal data. As a result, Chameleon can both reason over, as well as generate, arbitrary mixed-modal documents. Text tokens are represented in green and image tokens are represented in blue.

(a scheme from the [Chameleon paper](#) by Meta)

So far we've mostly discussed it theoretically. But now we can actually try it in Google AI Studio. Even though it's far from perfect, it's an exciting and important step towards native multimodal generations. I hope to see more of this stuff!

For now, I'd say that Gemini is better for clear, local changes, like giving a person a guitar, marking or removing objects. (Not flawless though.) Styling may be ok or not, depending on the style. I have a feeling that queries that require global changes of an

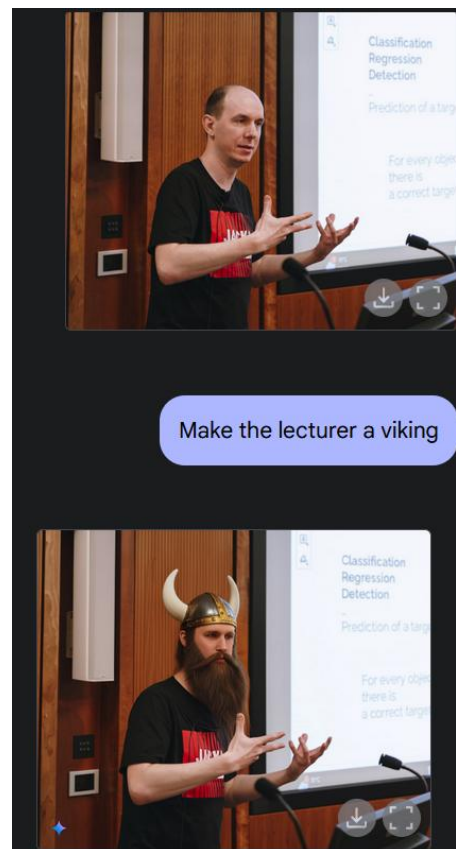


image (like “draw them like a heavy metal band”) isn’t Gemini’s forte. (And I don’t really blame the model for it; such queries are complicated.)



<https://www.alphaxiv.org/>

This is a space for exploring arxiv papers. And it boasts several cool AI features such as:

- Asking an LLM about a particular fragment in a paper: highlight and query!
- Automatic generation of short explanations of papers, even with generated illustrations! Maybe you won’t even need this paperwatch soon...

Not to mention that they also have paper discussion interface.

Gemini Robotics

<https://deepmind.google/discover/blog/gemini-robotics-brings-ai-into-the-physical-world/>

Gemini Robotics is a Gemini 2.0-based model designed for robotics. Just check the post, it showcases some very exciting perspectives.

Large Language Diffusion Models

<https://arxiv.org/pdf/2502.09992>

Most LLMs are decoder-only transformers, and as such they generate text autoregressively, token by token, which means – they “don’t know” what’ll be in the end when they start generation. On the other hand, having a whole resulting-text-spanning plan before coming to details might be beneficial – and this can be done with clever

orchestration, of course, but it would be so cool if LLMs could get rid of autoregressive principle!

This paper, inspired by diffusion models which still rule in the vision generation domain, the authors make an attempt at challenging autoregression.

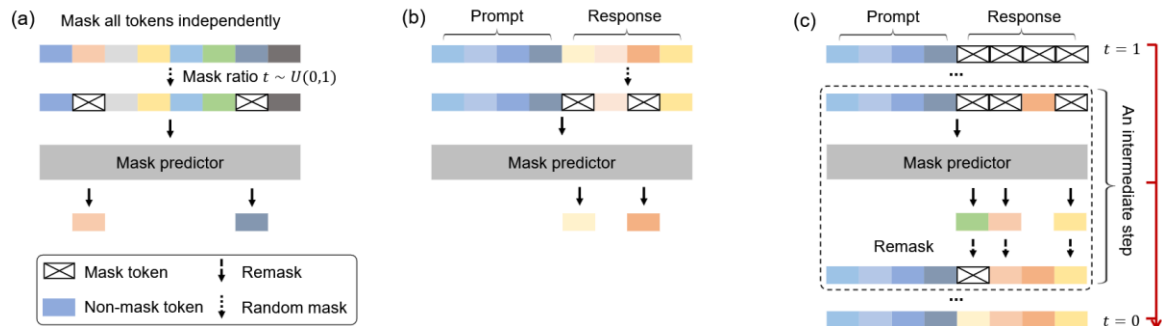


Figure 2. A Conceptual Overview of LLaDA. (a) Pre-training. LLaDA is trained on text with random masks applied independently to all tokens at the same ratio $t \sim U[0, 1]$. (b) SFT. Only response tokens are possibly masked. (c) Sampling. LLaDA simulates a diffusion process from $t = 1$ (fully masked) to $t = 0$ (unmasked), predicting all masks simultaneously at each step with flexible remask strategies.

The idea is to train the LLM for masked prediction (say hi to BERT), but to do it in a clever, two-stage way:

Stage 1: Pre-training. On this stage, a model learns to predict randomly placed masked tokens. Akin to LLM pre-training, this doesn't teach the model to give answers to queries, but instead shows it how language works.

Stage 2: Instruction tuning. This time, the model is trained on pairs (instruction, answer), and only answer tokens are masked.

No diffusion so far – but the analogy starts to work for the inference (sampling) process, which goes in several steps $1 = t_0 > t_1 > \dots > t_N = 0$:

- At $t_0 = 1$, all answer tokens are masked,
- At t_i , whatever is masked, is predicted; after that $\frac{t_{i+1}}{t_i}$ that were masked at t_i are remasked.
- At $t_N = 0$, the final answer is obtained.

The main strategy of choosing tokens to remask is low-confidence remasking: those tokens that get low confidence, get another chance to be predicted, given more tokens.

Also, semi-autoregressive sampling may be used, with the answer predicted in consecutive blocks, where inside each of them the above “diffusion process” is used.

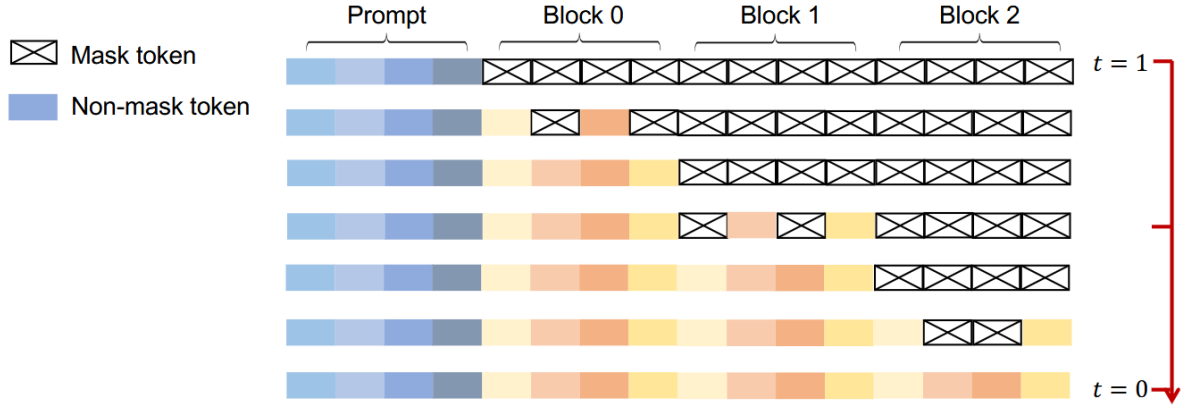


Figure 4. A Conceptual Overview of the Semi-autoregressive Sampling.

The authors train two models: LLaDA 1B and LLaDA 8B, and compare them with LLaMA 8B and smaller autoregressive baselines pre-trained and SFT-ed alongside LLaDA.

The results (see next page) are interesting, especially for math. At the same time, the model copes easily with natively non-autoregressive tasks. The authors demonstrate this on the *reverse poem generation* task. Specifically, they construct a dataset of 496 famous Chinese poem sentence pairs. Given a sentence from a poem, models are tasked with generating the subsequent line (forward) or the preceding line (reversal) without additional fine-tuning. And here are the results:

Table 3. Comparison in the Poem Completion Task.

	Forward	Reversal
GPT-4o (2024-08-06)	82.7	34.3
Qwen2.5 7B Instruct	75.9	38.0
LLaDA 8B Instruct	48.8	42.4

And it looks like in this broken-order task LLaDA excels.

Ok, now the benchmarks:

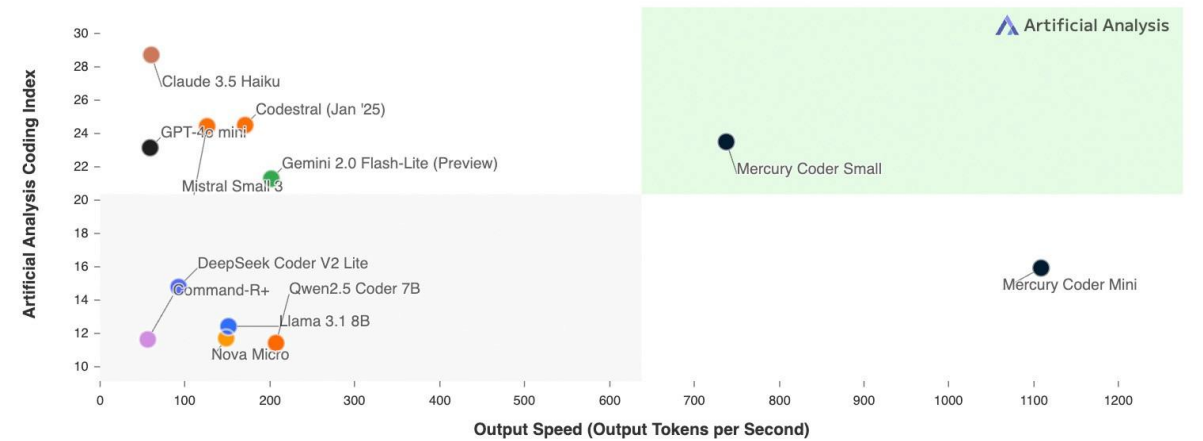
	LLaDA 8B*	LLaMA3 8B*	LLaMA2 7B*	Qwen2 7B†	Qwen2.5 7B†	Mistral 7B†	Deepseek 7B¶
Model	Diffusion	AR	AR	AR	AR	AR	AR
Training tokens	2.3T	15T	2T	7T	18T	-	2T
General Tasks							
MMLU	65.9 (5)	65.4 (5)	45.9 (5)	70.3 (5)	74.2 (5)	64.2 (5)	48.2 (5)
BBH	49.8 (3)	57.6 (3)	37.3 (3)	62.3 (3)	70.4 (3)	56.1 (3)	39.5 (3)
ARC-C	47.9 (0)	53.1 (0)	46.3 (0)	60.6 (25)	63.7 (25)	60.0 (25)	48.1 (0)
Hellaswag	72.5 (0)	79.1 (0)	76.0 (0)	80.7 (10)	80.2 (10)	83.3 (10)	75.4 (0)
TruthfulQA	46.4 (0)	44.0 (0)	39.0 (0)	54.2 (0)	56.4 (0)	42.2 (0)	-
WinoGrande	74.8 (5)	77.3 (5)	72.5 (5)	77.0 (5)	75.9 (5)	78.4 (5)	70.5 (0)
PIQA	74.4 (0)	80.6 (0)	79.1 (0)	-	-	-	79.2 (0)
Mathematics & Science							
GSM8K	70.7 (4)	53.1 (4)	14.3 (4)	80.2 (4)	85.4 (4)	36.2 (4)	17.4 (8)
Math	27.3 (4)	15.1 (4)	3.2 (4)	43.5 (4)	49.8 (4)	10.2 (4)	6.0 (4)
GPQA	26.1 (5)	25.9 (5)	25.7 (5)	30.8 (5)	36.4 (5)	24.7 (5)	-
Code							
HumanEval	33.5 (0)	34.2 (0)	12.8 (0)	51.2 (0)	57.9 (0)	29.3 (0)	26.2 (0)
HumanEval-FIM	73.8 (2)	73.3 (2)	26.9 (2)	-	-	-	-
MBPP	38.2 (4)	47.4 (4)	18.4 (4)	64.2 (0)	74.9 (0)	51.1 (0)	39.0 (3)
Chinese							
CMMLU	69.9 (5)	50.7 (5)	32.5 (5)	83.9 (5)	-	-	47.2 (5)
C-Eval	70.5 (5)	51.7 (5)	34.0 (5)	83.2 (5)	-	-	45.0 (5)

Here’s also one of the texts generated by LLaDA, with each token painted depending on the “diffusion” step after with the token doesn’t change (the darker, the later). One may persuade themselves that there’s a pattern. Feel free to find yours :) It would be interesting to experiment with this and find some data-verified patterns.

Sampling Process	
User	Lily can run 12 kilometers per hour for 4 hours. After that, she runs 6 kilometers per hour. How many kilometers can she run in 8 hours?
LLaDA	In4hours, Lily runs 12 * 4 = 48 kilometers . After 4 hours she runs 6 * 4 = 24 kilometers . In total she runs 48 + 24 = 72 kilometers in 8 hours . Result 72

By the way, Inception published their [diffusion LLM Mercury](#) and boast cool results on coding benchmarks.

	Mercury Coder Mini	Mercury Coder Small	Gemini 2.0 Flash-Lite	Claude 3.5 Haiku	GPT-4o Mini
Throughput (toks/sec)	1109	737	201	61	59
HumanEval	88.0	90.0	90.0	86.0	88.0
MBPP	77.1	76.6	75.0	78.0	74.6
EvalPlus	78.6	80.4	77.3	75.1	78.5
MultiPL-E	74.1	76.2	79.5	72.3	72.0
LiveCodeBench	17.0	25.0	18.0	31.0	23.0
BigCodeBench	42.0	45.5	44.4	45.4	46.8
Fill-in-the-Middle	82.2	84.8	60.1	45.5	60.9



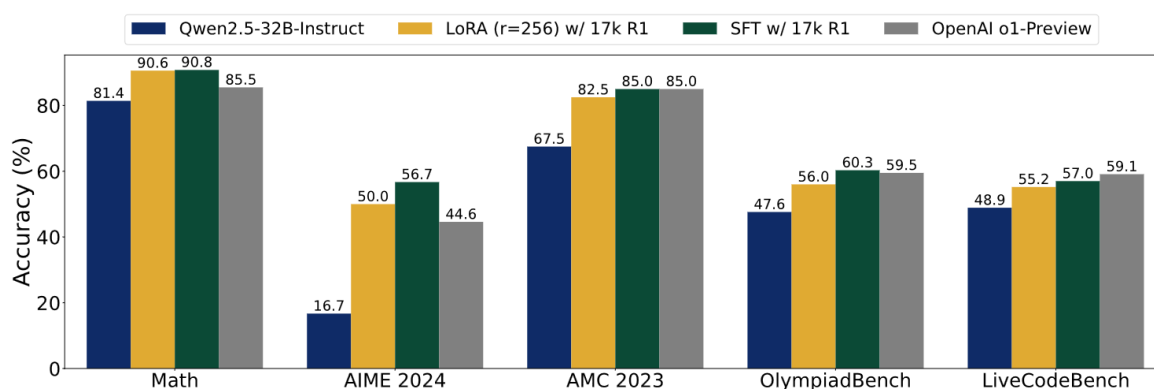
They also have a cool visualization of the generation process on the web site, so check it!

LLM Reasoning-related papers

LLMs Can Easily Learn to Reason from Demonstrations. Structure, not content, is what matters!

<https://arxiv.org/pdf/2502.07374>

Yet another paper about how to train a long-reasoning model efficiently. Compared to [LIMO](#) or [s1](#), the authors take a lot of long CoT data to fine tune Qwen2.5-32B-Instruct – **17k** instead of 1000 or less. But still, they get some new and interesting insights into the ideal composition of long CoT training data. And what is surprisingly cool – is that **they were able to achieve o1-preview performance** (well, in some sense) **by updating fewer than 5% parameters with LoRA fine-tuning**. That sounds huge.



(b) Performance of different models on five different reasoning benchmarks.

As data generators, the authors used DeepSeek R1 and QwQ-32B-Preview (two solid long reasoning models), choosing only solutions that led to correct answers. But apart from just training their model on this data, they also perform some very interesting experiments in order to understand what is more important: math correctness of individual steps or the overall logical structure.

Their experiments included introducing:

1. Wrong or Corrupted Local Content

- Wrong Answer Samples:** they trained the model on solutions that led to incorrect answers as well. This led to 3.2% less average accuracy - 63.1%.
- Digits Corrupted Samples:** correct solutions were corrupted by replacing each digit with a random number between 0 and 9. Potentially this might lead to nonsense such as “1+1=3”. The results were quite surprising:
 - With 70% of the digits corrupted, the average score only drops by 4.3%
 - However, when all digits are corrupted, the average performance plunges to 2.7% (to, not by!).

- c. **Reasoning Keyword Removal:** removing thought branching markers such as 'wait', 'let me think again', 'but',... Even after removing all the keywords, the average score only drops by 3.3%.

2. **Corrupted Global Reasoning Structure.** Here are the transformations used.

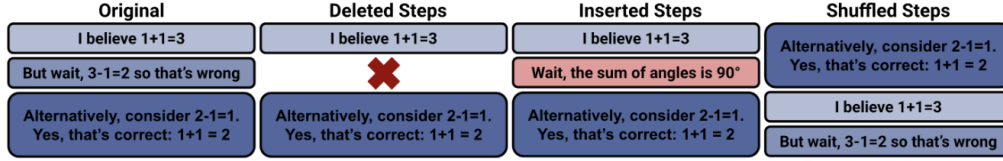


Figure 3: **Reasoning step modifications.** To evaluate perturbations to global structure across reasoning steps, we perform three modifications: deletion, insertion, and shuffling. These modifications break logical consistency across steps and degrade model accuracy far more than changes to local content within reasoning steps.

Now, corrupting reasoning structure leads to significantly worse results. This begs the hypothesis that for long CoT training having right global structure is much more important than local correctness. I'd say this in a sense aligns with the observations of the LIMO paper and with the data guidelines described there.

	MATH500	AIME24	AMC23	Olympiad.	Avg.
Baselines					
Original	84.8	16.7	67.5	47.6	56.7
Correct	89.2	40.0	77.5	58.5	66.3
Content Modifications					
Wrong Answers	88.6	30.0	77.5	56.1	63.1
Corrupted Digits					
100%	5.4	0.0	2.5	2.8	2.7
70%	85.6	30.0	77.5	54.8	62.0
50%	87.6	36.7	77.5	55.0	64.2
20%	88.4	30.0	82.5	57.2	64.5
Removed keywords					
100%	86.6	33.3	77.5	54.4	63.0
50%	87.6	36.7	82.5	56.7	65.9
20%	87.2	33.3	72.5	56.1	62.3
Structure Modifications					
Shuffled Steps					
100%	81.8	23.3	70.0	49.1	56.1
67%	82.0	26.7	72.5	47.6	57.2
33%	85.6	33.3	75.0	55.3	62.3
Deleted Steps					
100%	79.2	13.3	60.0	45.4	49.5
67%	84.2	26.7	55.0	48.1	53.5
33%	88.2	23.3	80.0	57.7	62.3
Inserted Steps					
100%	77.0	10.0	50.0	41.1	44.5
67%	81.8	20.0	60.0	46.0	52.0
33%	86.6	33.3	77.5	57.2	63.7

Cognitive Behaviors that Enable Self-Improving Reasoners, or, Four Habits of Highly Effective STaRs

<https://arxiv.org/pdf/2503.01307>

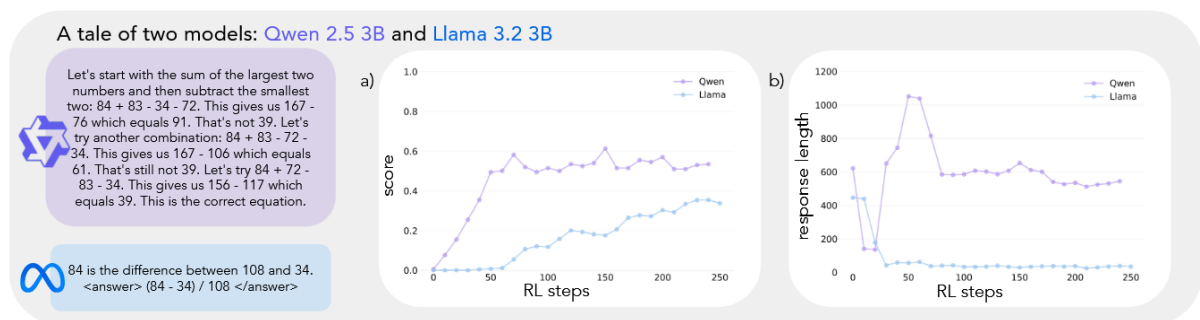
As the [Demystifying Long Chain-of-Thought Reasoning in LLMs](#) paper states

Core reasoning abilities, such as error correction, are inherently present in base (pre-trained) models—even if activating them can be tricky.

...but apparently, not in all base models.

The authors of this paper experimented with **Qwen-2.5-3B** and **Llama-3.2-3B**, training them with Reinforcement Learning to play Countdown, a game with the goal is to combine input numbers with arithmetic operations to reach a target number. This game requires non-linear planning and experimentation which makes it a good example.

And it turns out that Llama and Qwen behave very differently!



As you see, Llama learns much worse. But why? The authors further distinguish 4 reasoning patterns and observe if they are present in the pre-trained models

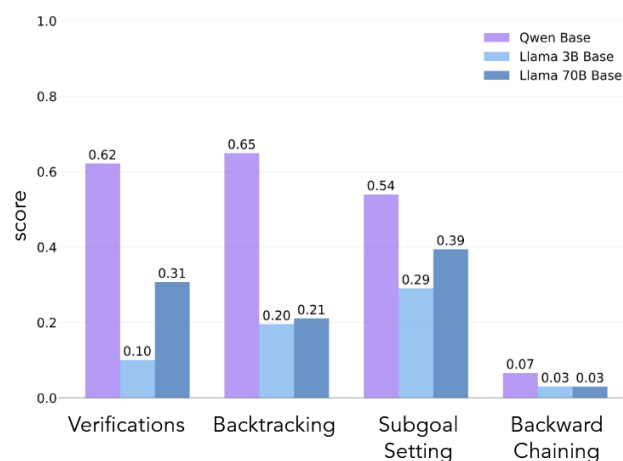
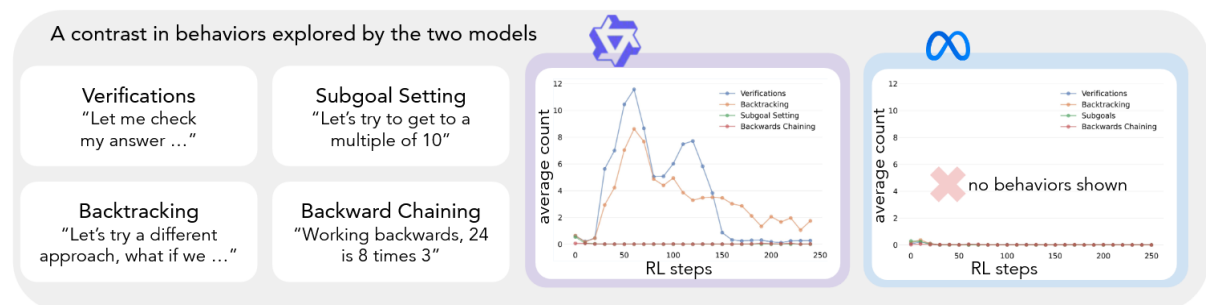


Figure 4: Exploration of different reasoning behaviors in base models. An analysis with Qwen2.5-3B, Llama3.2-3B, and Llama3.1-70B on Countdown.

... and how they evolve during RL training.



It begs the conclusion that RL is only able to amplify long-reasoning behaviours if they are substantially present in the pre-trained model. To justify this, the authors perform several checks.

First of all, they look at popular math pre-training datasets and observe that they are not very dense with the four non-linear reasoning patterns. (See the picture on the right.)

Then, they construct two SFT datasets – with non-linear reasoning and without it – and observe that after a targeted SFT, Llama is able to learn playing Countdown as well as Qwen.

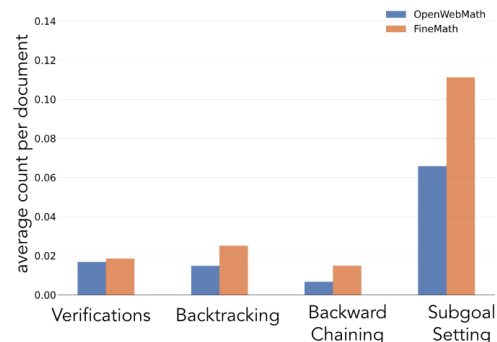


Figure 7: Behaviors present in Math Pretraining Datasets. An analysis of the behaviors present in 200,000 randomly sampled documents of OpenWebMath and FineMath. We measure the average count of the behaviors in each document.

So, one more illustration of the importance of pre-training data!

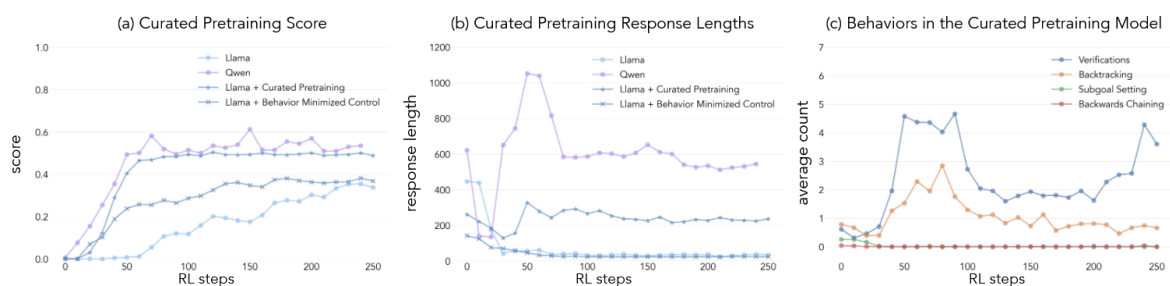


Figure 8: Impact of curated pretraining on model performance and behavior. (a) Comparison of performance scores across base models (Llama-3.2-3B, Qwen-2.5-3B) and Llama variants with curated pretraining versus behavior-minimized control. (b) Evolution of response lengths during training for each model configuration. (c) Emergence and development of specific cognitive behaviors in the curated pretraining model over training steps.

Reinforcement Learning is All You Need

<https://arxiv.org/pdf/2503.09512>

The single author of this paper also trains **Qwen-2.5-3B** to play Countdown with Reinforcement Learning. (Though with GRPO while the authors of the previous paper used PPO; I don't think it's very important anyway.)

Apart from observing first-hand the “aha moment” where the model discovers non-linear reasoning,

Aha Moment

```
<think> First, I can see that none of the numbers are divisible by 3 directly. So, I'll try to break this down using the given operations. </think>
<think> One approach could be to use the 84 and 16 to get a number close to 36 with a subtraction operation. </think>
<think> I can try subtracting 24 from 84, which gives us  $84 - 24 = 60$ . This is not exactly 36, but it's close. </think>
<think> Now, I need to get from 60 to 36. Let's use the 24 we worked with before, and we subtract 24 from 63 (from the list) to get 39. Then subtract 16 to get 18. </think>
<think> So, I need a way to get 18 from the remaining numbers. We can use 16 itself, since it's one of the given numbers, and we need to get to 18 from it. </think>
<think> I can add 2 to 16 to get 18, so our equation looks like:  $(84 - 24) - (63 - 61)$ . But wait, we need 61, not 63. </think>
<think> I realize I need to subtract 2 more from 63 to get 61. So, the final equation is:  $(84 - 24) - (63 - 2)$ . Let's calculate this. </think> <answer>  $(84 - 24) - (63 - 2) = 36$  </answer>
```

...the author checks whether training with RL for only the Countdown problem is able to change the model's behavior on other tasks. And it seems, it can!

Table 4: Comparison of Model Performance across Different Tasks in BBH

Tasks	Base Model	Base Model + R1 Prompt	Trained Model
Group Result	0.375	0.365	0.4406
Boolean Expressions	0.8280	0.7840	0.9040
Causal Judgement	0.4439	0.4492	0.4920
Date Understanding	0.3560	0.2480	0.4920
Disambiguation QA	0.4200	0.3800	0.4960
Dyck Languages	0.0080	0.0240	0.0040
Formal Fallacies	0.4720	0.3440	0.5040
Geometric Shapes	0.0800	0.0520	0.1280
Hyperbaton	0.5520	0.5960	0.5560
Logical Deduction (Five Objects)	0.3440	0.3240	0.3360
Logical Deduction (Seven Objects)	0.2680	0.3160	0.3440
Logical Deduction (Three Objects)	0.3080	0.2320	0.6200
Movie Recommendation	0.4760	0.3520	0.3760
Multistep Arithmetic (Two)	0.1760	0.2000	0.6000
Navigate	0.7480	0.7600	0.7200
Object Counting	0.0520	0.0400	0.1200
Penguins in a Table	0.6712	0.7055	0.8014
Reasoning about Colored Objects	0.5000	0.5240	0.6840
Ruin Names	0.3560	0.3480	0.2760
Salient Translation Error Detection	0.1920	0.2120	0.4040
Snarks	0.5000	0.5562	0.5955
Sports Understanding	0.5560	0.5400	0.4920
Temporal Sequences	0.3040	0.3360	0.2360
Tracking Shuffled Objects (Five Objects)	0.3720	0.3360	0.2680
Tracking Shuffled Objects (Seven Objects)	0.2200	0.2320	0.1840
Tracking Shuffled Objects (Three Objects)	0.3760	0.4000	0.6320
Web of Lies	0.6560	0.5400	0.7400
Word Sorting	0.1000	0.1240	0.1000

[BigBenchHard](#) is quite an old benchmark, but still that's an interesting comparison.

New attention mechanisms

Self-attention is cool, but its power is somewhat undermined by its quadratic complexity (quadratic as a function of context length) and memory usage (for KV cache). So, researchers don't ever stop suggesting new optimization of the self-attention mechanism in LLMs or even its alternatives. As one of those, **state space models** still hold a special place in my heart, even though researchers seem to have become disillusioned with them. However, though, **AI21 labs recently launched Jamba 1.6**, their new hybrid model with alternating self-attention and SSM blocks. The model isn't especially striking in terms of quality, but it's nice that someone still places some hope on SSMs.

Ok, but enough nostalgia; let's briefly check what appeared recently.

Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention

by DeepSeek <https://arxiv.org/pdf/2502.11089>

The authors suggest using three sparse attention mechanisms with the final attention output being equal to their combination with grained gating g_t^c :

$$o_t = \sum_{c \in C} g_t^c \cdot \text{Attn}(q_t, \tilde{K}_t^c, \tilde{V}_t^c)$$

When they say “sparse”, they mean that none of the mechanisms works with all the keys and values, which reduces quadratic complexity to something lower.

The three mechanisms are:

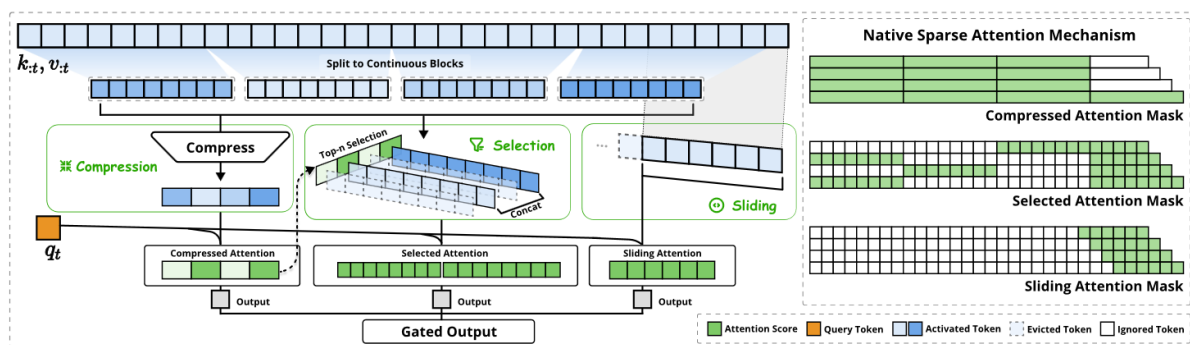


Figure 2 | Overview of NSA's architecture. Left: The framework processes input sequences through three parallel attention branches: For a given query, preceding keys and values are processed into compressed attention for coarse-grained patterns, selected attention for important token blocks, and sliding attention for local context. Right: Visualization of different attention patterns produced by each branch. Green areas indicate regions where attention scores need to be computed, while white areas represent regions that can be skipped.

1. Attention mechanism based on token compression

This mechanism compresses keys and values in continuous blocks of length l using some learnable function φ , so that when processing the kl -th token the keys used by this attention mechanism are:

$$\tilde{K}_{tl}^{cmp} = [\varphi(k_0, \dots, k_{l-1}), \quad \varphi(k_l, \dots, k_{2l-1}), \quad \varphi(k_{(t-1)l}, \dots, k_{tl-1})]$$

and the same with values.

This, of course, resembles many block approaches we've seen in the past, like [Infini-attention](#). Those approaches also tried to compress memory (i.e. keys and values) into something smaller than the full KV cache and later use it.

2. **Attention mechanism based on token selection** selectively preserves keys and values from the past to be used at attention computation for the t -th token. (Others are ignored.) Unlike (1), keys and values inside each selected block are used individually.

Tokens are selected in blocks (as in the picture above), because it makes things easier for the GPU.

The question is where to get importance weights for block selection, but luckily we can use

$$p_t = \text{softmax}(q_t^T \tilde{K}_t^{cmp}),$$

where $\tilde{K}_t^{cmp}$ is the set of key vectors created by the token compression mechanism. Each key vector in $\tilde{K}_t^{cmp}$ corresponds to one block, so p_t is a set of block-wise “importance weights”. Top n “most important blocks” are used for attention computation by the second mechanism.

3. **Sliding window attention mechanism** is just the familiar sliding window attention.

I would say that (1) and (3) are quite familiar to me while (2) is something interesting. Now, an important thing: while many other attention sparsification schemes are put to work at inference (or fine tuning), the authors of this approach underscore that sparse attention should be native – i.e., an LLM should be pre-trained from start with these architectural changes. So, all φ and g_t^c are also pre-trained.

Model	SQA			MQA				Synthetic		Code	Avg.
	MFQA-en	MFQA-zh	Qasper	HPQ	2Wiki	GovRpt	Dur	PassR-en	PassR-zh	LCC	
H2O	0.428	0.429	0.308	0.112	0.101	0.231	0.208	0.704	0.421	0.092	0.303
InfLLM	0.474	0.517	0.356	0.306	0.250	0.277	0.257	0.766	0.486	0.143	0.383
Quest	0.495	0.561	0.365	0.295	0.245	0.293	0.257	0.792	0.478	0.135	0.392
Exact-Top	0.502	0.605	0.397	0.321	0.288	<u>0.316</u>	0.291	0.810	0.548	0.156	0.423
Full Attn	0.512	<u>0.623</u>	<u>0.409</u>	<u>0.350</u>	<u>0.305</u>	0.324	<u>0.294</u>	<u>0.830</u>	0.560	<u>0.163</u>	<u>0.437</u>
NSA	<u>0.503</u>	0.624	0.432	0.437	0.356	0.307	0.341	0.905	<u>0.550</u>	0.232	0.469

Table 2 | Performance comparison between our NSA and baselines on LongBench, including subsets in single document QA, multi-document QA, synthetic and code task categories. NSA outperformed most of the baselines including Full Attention.

Here are some of the results on popular Q&A benchmarks. While most sparsification approaches behave significantly worse than fill self-attention, he authors demonstrate the superior performance of their method, at least on tasks that require reasoning and, as consequence, longer generation.

The authors also create an efficiently parallelized Cuda implementation and demonstrate huge speed-up compared to FlashAttention-2:

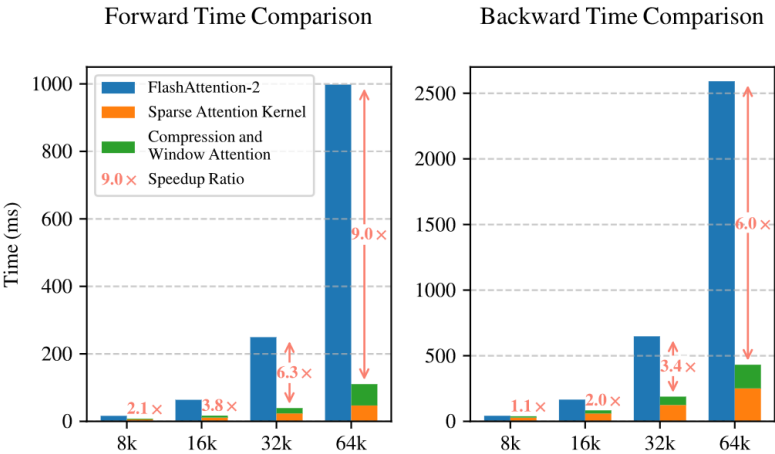


Figure 6 | Comparison of Triton-based NSA kernel with Triton-based FlashAttention-2 kernel. Our implementation significantly reduces latency across all context lengths, with the improvement becoming more pronounced as input length increases.

They also measure memory access speed, which is important because autoregressive generation is a memory-intensive, not compute-intensive operation. (Generation latency is mostly determined by the speed of moving things around memory.)

Context Length	8192	16384	32768	65536
Full Attention	8192	16384	32768	65536
NSA	2048	2560	3584	5632
Expected Speedup	4×	6.4×	9.1×	11.6×

Table 4 | Memory access volume (in equivalent number of tokens) per attention operation during decoding. Due to the low arithmetic intensity and memory-bound nature of decoding, the expected speedup is approximately linear with the volume of memory access.

The model they train is a 27B Mixture-of-Expert model with MoE based on DeepSeek’s architecture, with 3B parameters active for each token at inference.

Whether I believe that this approach will bring value, is a difficult question. Standard full attention is so cool and so greatly optimized, that it’s very hard to compete with it. Moreover, full attention is time-tested, while any new approach needs to be validated much more thoroughly than with a single paper’s tests, otherwise we may be testing the wrong things in the wrong way, as we do quite often. So, let’s wait and see.

The FFT Strikes Back: An Efficient Alternative to Self-Attention

<https://arxiv.org/pdf/2502.18394v1>

To be accurate, this paper is not about LLMs. It's more about classification transformers (well, LLMs are also classifiers, but the authors don't venture into this). The main ingredient in this paper is **Fast Fourier Transform**. So far, it was actively used as a device for speeding up long convolutions, due to the fact that

$$W * X = IFFT(FFT(W) \odot FFT(X)),$$

where IFFT is the Inverse Fast Fourier Transform, $\odot$ is an elementwise product. If the kernel W is really long – not like 3×3 convolutions in CV, but, say, the length n of the whole sequence X , this really gives a speed-up. Indeed, performing convolution in a naïve way would be $O(n^2)$ while FFT does the job in $O(n \log n)$, while elementwise product is almost free. This is, for example, leveraged in State Space Models, which are at the same time both recurrent and convolutional networks.

Now, this paper suggests something like that as a replacement for the self-attention mechanism. Let's briefly summarize their approach. Let X be the initial sequence of length n . We do the following:

1. Compute FFT of X along the token dimension: $F = FFT(X)$,
2. Apply **adaptive spectral filtering** to emphasize important “frequencies”.

First, compute a global context vector

$$c = \frac{1}{n} \sum_{i=1}^n X_i$$

and pass it through a trainable MLP to obtain

$$\Delta W = MLP(c)$$

Now, the final “filter” is

$$W = W_{base} + \Delta W,$$

Where W_{base} is a constant filter. Now,

$$\tilde{F} = F \odot W$$

3. Apply non-linear activations for capturing higher-order relationships in the complex frequency domain. Basically, it's a complex analog of ReLU. (The result of FFT is a complex array.) If $z = r e^{i\varphi}$, then

$$\text{modReLU}(z) = \begin{cases} (r + b) e^{i\varphi}, & \text{if } r + b > 0 \\ 0, & \text{otherwise} \end{cases}$$

4. The final output is $Y = IFFT(\text{modReLU}(\tilde{F}))$.

The most computationally intensive operations here are FFT and IFFT, and they are both $O(n \log n)$, where n is the sequence length, so it's more efficient than self-attention.

You probably wonder what it has to do with attention. But let's recall that the main purpose of self-attention is **mixing information between different tokens**. Each “frequency feature” gathers information from the whole sequence. So even though the transformations we do on steps 2 and 3 work with different features independently, they

are in a sense global transformations and somewhat lead to disseminating of information along the sequence.

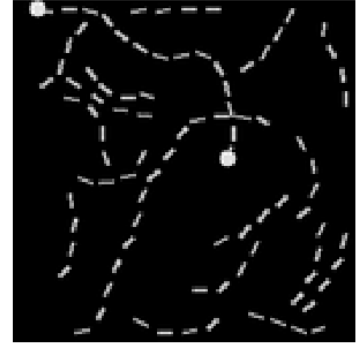
The authors try their method on visual classification including, for example, the Pathfinder challenge requiring a model to understand whether two points represented as circles are connected by a path consisting of dashes. (See a picture on the right.) The results seem to be good:

Model	ListOps	Text	Retrieval	Image	Pathfinder	Path-X	Avg.
Transformer	36.06	61.54	59.67	41.51	80.38	OOM	55.83
FNet	35.33	65.11	59.61	38.67	77.80	FAIL	55.32
FFTNet (ours)	37.65	66.01	60.21	42.02	80.71	83.25	58.31

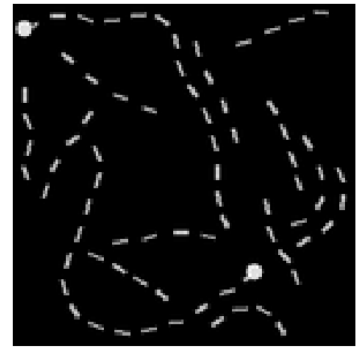
Table 1: Accuracy (%) on the Long Range Arena (LRA) benchmark. OOM indicates out-of-memory and FAIL indicates the model could not process the dataset. Our **FFTNet** obtains the best average accuracy.

And, of course, the latency of such model will be, as the context size grows, much better than transformer models have.

I'd say that the middle part of the pipeline (steps 2 and 3) are somewhat lacking, like there should be more interaction between frequencies. But anyway; let's see if anyone dares apply FFT for LLMs.



(a) A positive example.



(b) A negative example.

From RAG to Memory: Non-Parametric Continual Learning for Large Language Models

<https://arxiv.org/pdf/2502.14802>

As LLMs' native long-term memory isn't so far immaculate, RAG is needed to help with that. One of the most popular RAG schemes involves vector databases created in hope that, for good embeddings, geometric proximity corresponds to semantic closeness. However, vector search works in semantic associations and may fail to retrieve particular facts or relations. So, many other approaches are tried, including graph databases which explicitly encode relations.

The authors continue the efforts started in the [HippoRAG](#) paper and suggest a **HippoRAG 2** system.

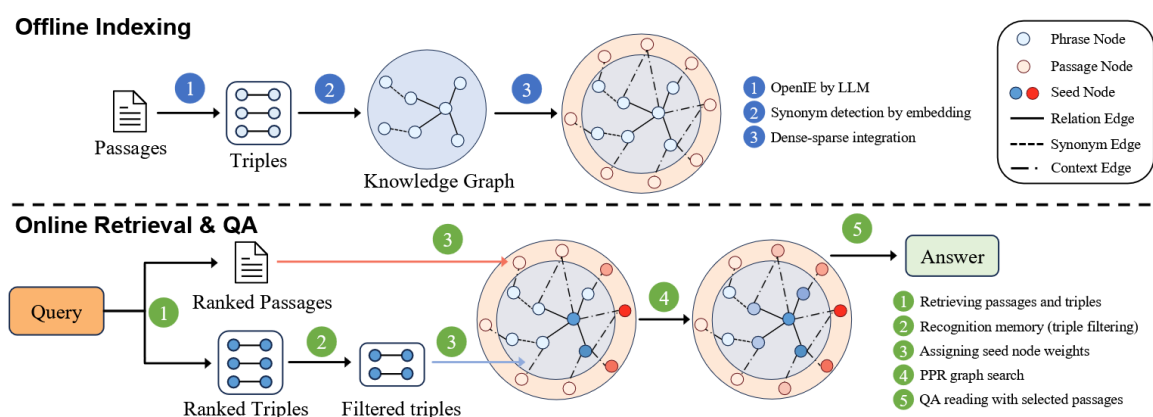


Figure 2. HippoRAG 2 methodology. For offline indexing, we use an LLM to extract open KG triples from passages, with synonym detection applied to phrase nodes. Together, these phrases and passages form the open KG. For online retrieval, an embedding model scores both the passages and triples to identify the seed nodes of both types for the Personalized PageRank (PPR) algorithm. Recognition memory filters the top triples using an LLM. The PPR algorithm then performs context-based retrieval on the KG to provide the most relevant passages for the final QA task. The different colors shown in the KG nodes above reflect their probability mass; darker shades indicate higher probabilities induced by the PPR process.

At indexing stage, a knowledge graph (KG) is created:

- From each text (“**passage**”) a number of **triples** (2 objects - **phrases** - & **relation**) is extracted. From each of them, two **phrase nodes** and a **relation edge** are created.
- Also, special efforts are done to identify synonymous phrases, creating **synonym edges** in the knowledge graph.
- Finally, in addition to the “sparse” relation and synonym information, “dense” context data is also introduced into KG as **passage nodes** connected by **context edges** to the phrases contained in it.

At retrieval stage:

- Relevant passage nodes are retrieved using vector search,
- Relevant triples are retrieved from query, also using vector search. Then the set of triples is refined (decreased) using an LLM.

- Finally, PPR (Personalized PageRank, see [this paper](#)) algorithm is used from the seed nodes constructed above to find more relevant nodes. Eventually, the selected passages are used for answer generation.

Here are the results:

Table 2. **QA performance** (F1 scores) on RAG benchmarks using Llama-3.3-70B-Instruct as the QA reader. No retrieval means evaluating the parametric knowledge of the readers. HippoRAG (and HippoRAG 2) uses Llama-3.3-70B-Instruct as the extractor (and the triple filter) and NV-Embed-v2 as the retriever. This table, along with the following ones, highlight the **best** and second-best results.

Retrieval	Simple QA		Multi-Hop QA				Discourse Understanding	Avg
	NQ	PopQA	MuSiQue	2Wiki	HotpotQA	LV-Eval	NarrativeQA	
Simple Baselines								
None	54.9	32.5	26.1	42.8	47.3	6.0	12.9	38.4
Contriever (Izacard et al., 2022)	58.9	53.1	31.3	41.9	62.3	8.1	19.7	46.9
BM25 (Robertson & Walker, 1994)	59.0	49.9	28.8	51.2	63.4	5.9	18.3	47.7
GTR (T5-base) (Ni et al., 2022)	59.9	56.2	34.6	52.8	62.8	7.1	19.9	50.4
Large Embedding Models								
GTE-Qwen2-7B-Instruct (Li et al., 2023)	62.0	56.3	40.9	60.0	71.0	7.1	21.3	54.9
GritLM-7B (Muennighoff et al., 2024)	61.3	55.8	44.8	60.6	73.3	9.8	23.9	56.1
NV-Embed-v2 (7B) (Lee et al., 2025)	61.9	55.7	45.7	61.5	75.3	9.8	25.7	57.0
Structure-Augmented RAG								
RAPTOR (Sarathi et al., 2024)	50.7	56.2	28.9	52.1	69.5	5.0	21.4	48.8
GraphRAG (Edge et al., 2024)	46.9	48.1	38.5	58.6	68.6	11.2	23.0	49.6
LightRAG (Guo et al., 2024)	16.6	2.4	1.6	11.6	2.4	1.0	3.7	6.6
HippoRAG (Gutiérrez et al., 2024)	55.3	55.9	35.1	71.8	63.5	8.4	16.3	53.1
HippoRAG 2	63.3	56.2	48.6	71.0	75.5	12.9	25.9	59.8

Looks nice.

I'll be frank, I'm not a huge fan of knowledge graphs, which are prone to going out of control in real-life situations. However, in some settings this approach may be viable. And, of course, it underscores the fact that vector search is not always the best thing to do.